



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION LAB WEEK 08

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d





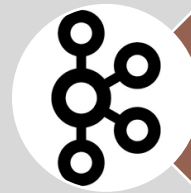
Week 8



Exploring a
Secondary Sort
Example



Importing Data with
Sqoop



Kafka Ordering
System Example



Lab Task 4



EXPLORING A SECONDARY SORT EXAMPLE

Exploring a Secondary Sort Example

- You'll run a MapReduce job in different ways to see the effects of various components in a secondary sort program.
- The program accepts lines in the form:
lastname firstname birthdate.
- The goal is to identify the youngest person with each last name.

Files and Directories Used in this Exercise

Eclipse project: `secondarysort`

Data files:

`~/training_materials/developer/data/nameyeartestdata`

Exercise directory: `~/workspace/secondarysort`

Exploring a Secondary Sort Example

Example Input:

Murphy Joanne 1963-08-12
Murphy Douglas 1832-01-20
Murphy Alice 2004-06-20

Expected Output:

Murphy Alice 2004-06-02

Note:

- All codes in this exercise has been built in the given project.
- Following the steps below you are going to progressively add each component to the job to accomplish the final goal.

Exploring a Secondary Sort Example

- Build the Program

1. In Eclipse, review but do not modify the code in the `secondarysort` project `example` package.
2. In particular, note the `NameYearDriver` class, in which the code to set the partitioner, sort comparator, and group comparator for the job is commented out. This allows us to set those values on the command line instead.
3. Export the jar file for the program as `secsort.jar`.
4. A small test datafile called `nameyeartestdata` has been provided for you, located in the secondary sort project folder. Copy the datafile to HDFS, if you did not already do so in the Writables lab.

Exploring a Secondary Sort Example

- Run as a Map-only Job.

5. The Mapper for this job constructs a composite key using the `StringPairWritable` type. See the output of just the mapper by running this program as a Map-only job:

```
$ hadoop jar secsort.jar example.NameYearDriver -Dmapred.reduce.tasks=0 nameyeartestdata secsortout
```

6. Review the output. Note the key is a string pair of last name and birth year.

Exploring a Secondary Sort Example

- Run using the default Partitioner and Comparators.

7. Re-run the job, setting the number of reduce tasks to 2 instead of 0.
8. Note that the output now consists of two files; one each for the two reduce tasks. Within each file, the output is sorted by last name (ascending) and year (ascending). But it isn't sorted between files, and records with the same last name may be in different files (meaning they went to different reducers).

Exploring a Secondary Sort Example

- Run using the custom partitioner.

9. Review the code of the custom partitioner class: `NameYearPartitioner`.
10. Re-run the job, adding a second parameter to set the partitioner class to use:
`-Dmapreduce.partition.class=example.NameYearPartitioner`
11. Review the output again, this time noting that all records with the same last name have been partitioned to the same reducer.

However, they are still being sorted into the default sort order (name, year ascending). We want it sorted by name ascending/year descending.

Exploring a Secondary Sort Example

- Run using the custom sort comparator.

12. The `NameYearComparator` class compares Name/Year pairs, first comparing the names and, if equal, compares the year (in descending order; i.e. later years are considered “less than” earlier years, and thus earlier in the sort order.) Re-run the job using `NameYearComparator` as the sort comparator by adding a third parameter:

```
-D mapred.output.key.comparator.class=  
example.NameYearComparator
```

13. Review the output and note that each reducer’s output is now correctly partitioned and sorted.

Exploring a Secondary Sort Example

- Run with the NameYearReducer.

14. So far we've been running with the default reducer, which is the Identity Reducer, which simply writes each key/value pair it receives. The actual goal of this job is to emit the record for the *youngest person* with each last name. We can do this easily if all records for a given last name are passed to a single reduce call, sorted in descending order, which can then simply emit the first value passed in each call.

15. Review the NameYearReducer code and note that it emits

16. Re-run the job, using the reducer by adding a fourth parameter:

```
-Dmapreduce.reduce.class=example.NameYearReducer
```

Alas, the job still isn't correct, because the data being passed to the reduce method is being grouped according to the full key (name and year), so multiple records with the same last name (but different years) are being output. We want it to be grouped by name only.

Exploring a Secondary Sort Example

- Run with the custom group comparator.

17. The `NameComparator` class compares two string pairs by comparing only the name field and disregarding the year field. Pairs with the same name will be grouped into the same reduce call, regardless of the year. Add the group comparator to the job by adding a final parameter:

```
-Dmapred.output.value.groupfn.class=  
example.NameComparator
```

18. Note the final output now correctly includes only a single record for each different last name, and that that record is the youngest person with that last name.



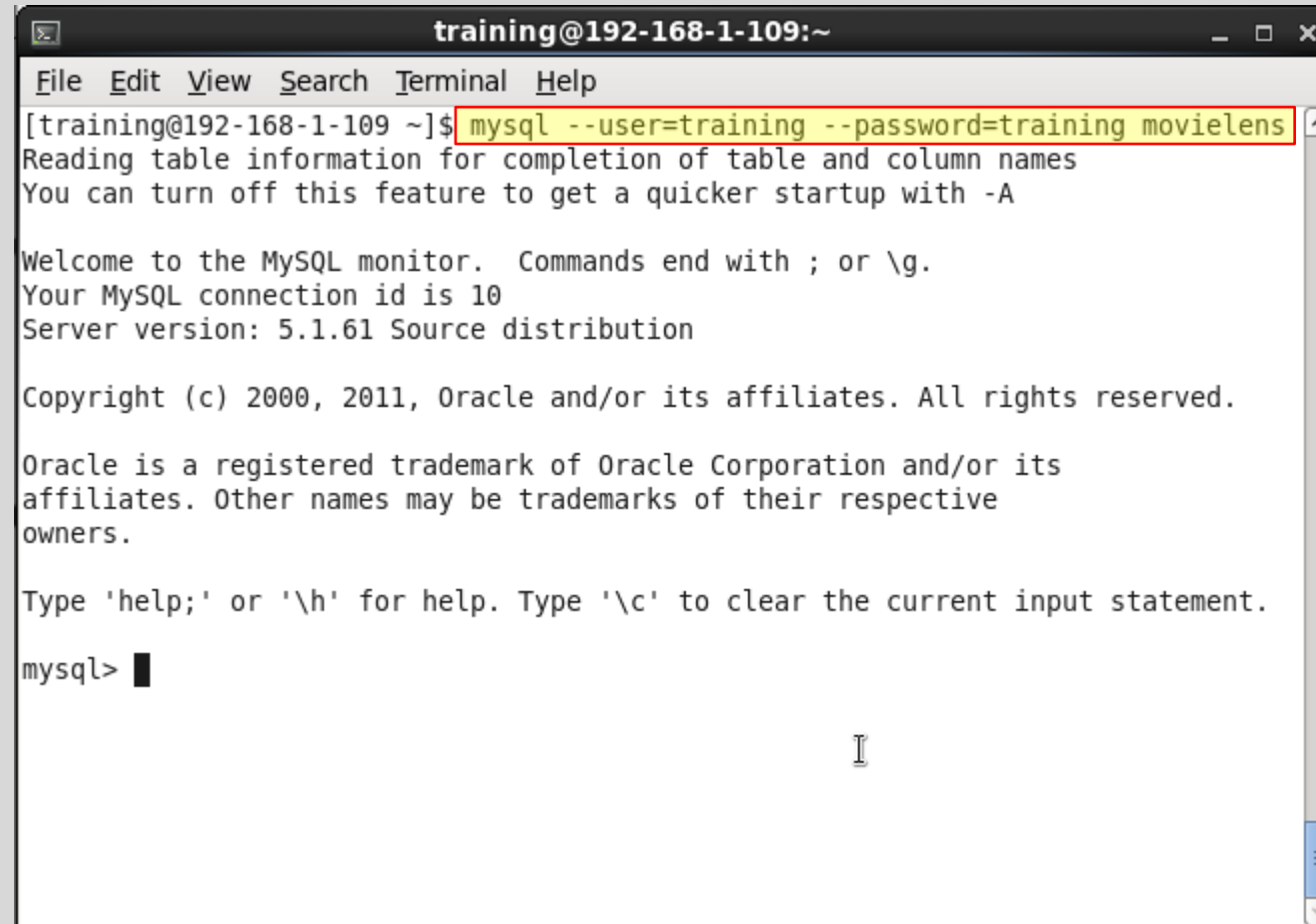
IMPORTING DATA WITH SQOOP

Importing Data with Sqoop

- In this exercise, you'll import data from a relational database using Sqoop.
- The data you load here will be used in the future labs and assignments.
- Consider the MySQL database *movielens*, derived from the MovieLens project of several related tables, but we will import only two of these: *movie*, which contains about 3,900 movies; and *movierating*, which has about 1,000,000 ratings of those movies.

Review the Database Tables

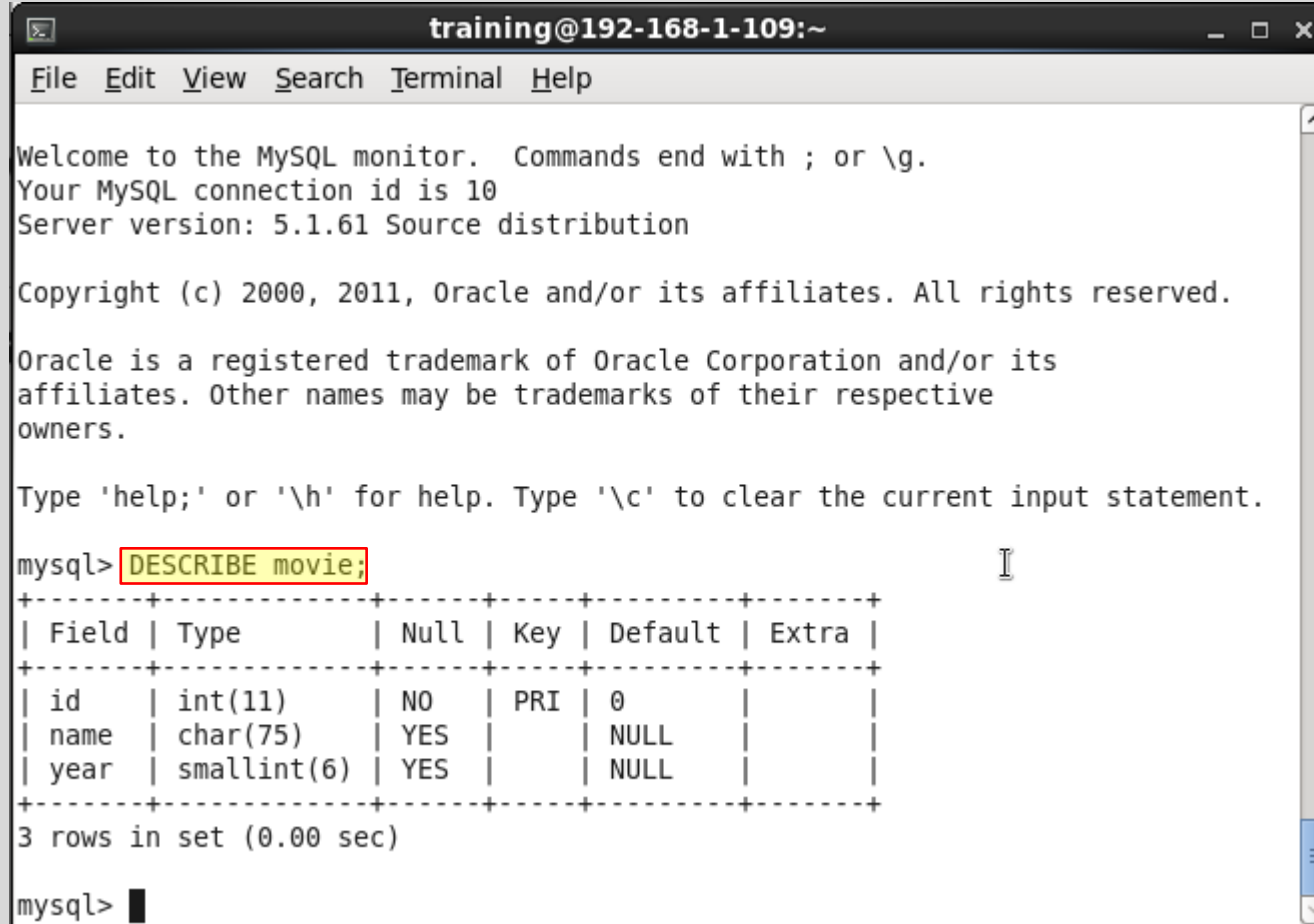
- Log on to MySQL:

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command 'mysql --user=training --password=training movielens' is entered and highlighted with a red box. The output shows MySQL version 5.1.61 and the 'mysql>' prompt.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ mysql --user=training --password=training movielens  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 10  
Server version: 5.1.61 Source distribution  
  
Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql> █
```

Review the Database Tables

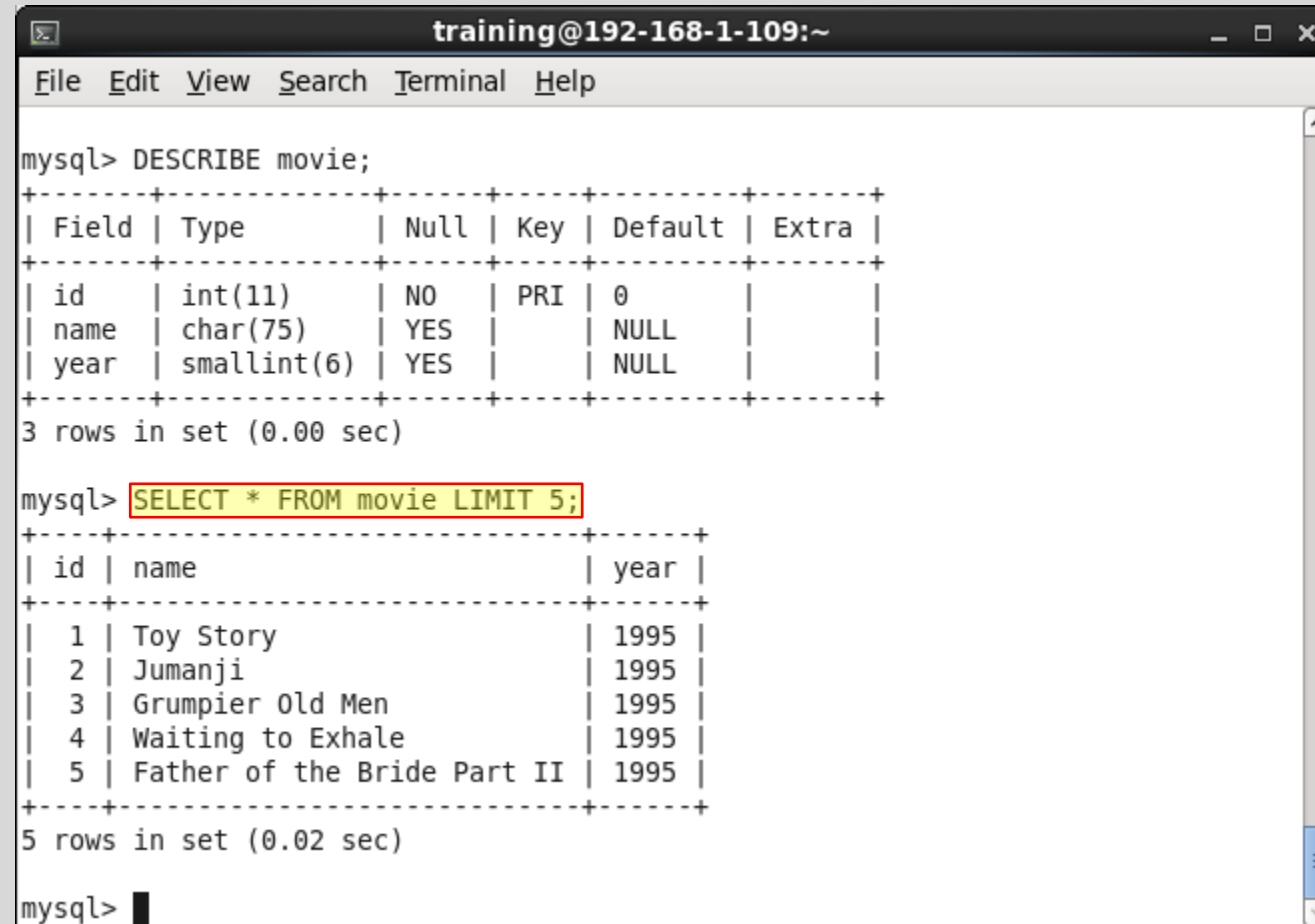
- Review the structure and contents of the *movie* table:



```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 10  
Server version: 5.1.61 Source distribution  
  
Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql> DESCRIBE movie;  
+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+  
| id    | int(11)       | NO   | PRI | 0        |       |  
| name  | char(75)      | YES  |     | NULL     |       |  
| year  | smallint(6)   | YES  |     | NULL     |       |  
+-----+  
3 rows in set (0.00 sec)  
  
mysql> █
```

Review the Database Tables

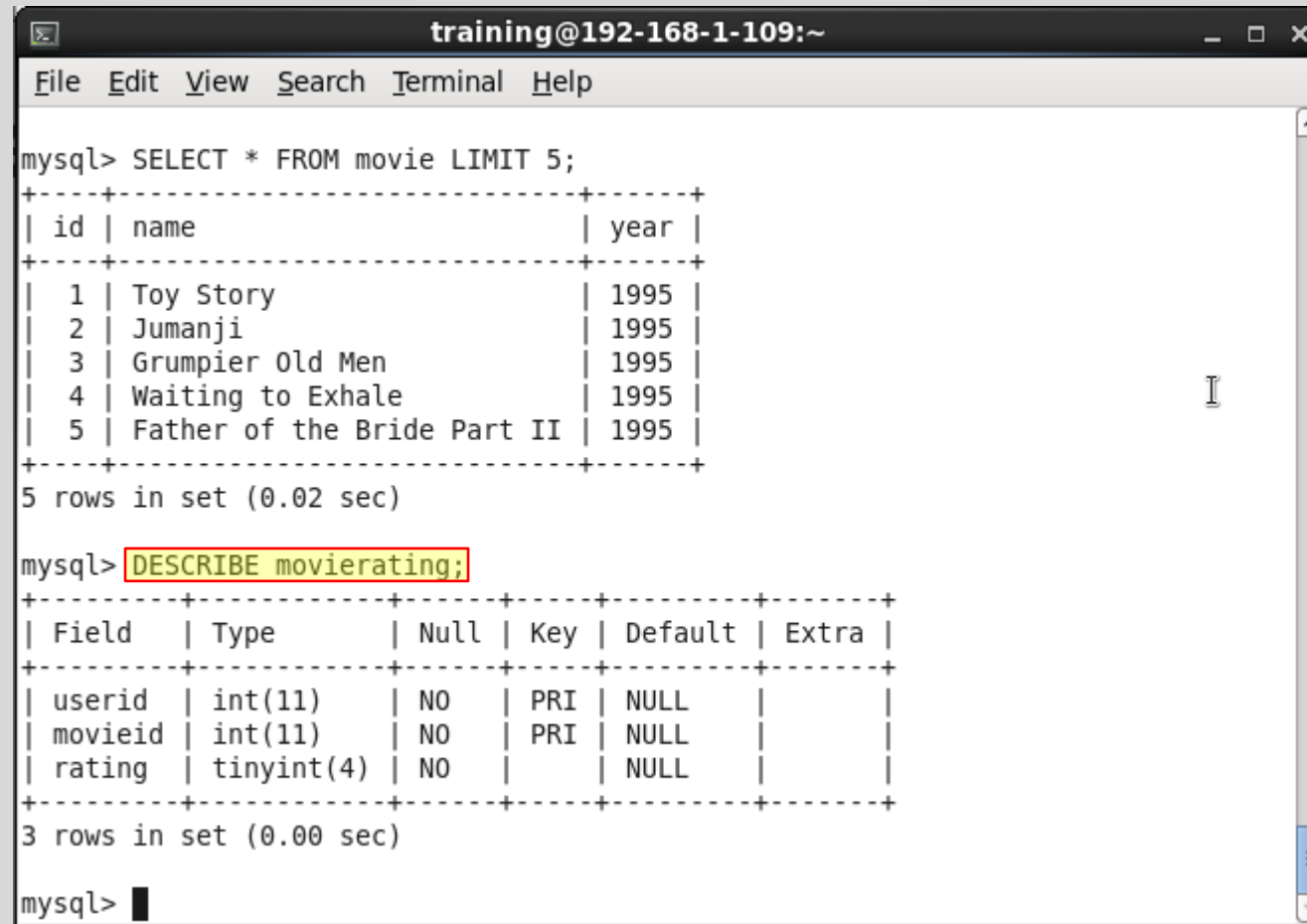
- Review the structure and contents of the *movie* table:



```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
mysql> DESCRIBE movie;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| id    | int(11)       | NO   | PRI | 0        |       |  
| name  | char(75)      | YES  |     | NULL     |       |  
| year  | smallint(6)   | YES  |     | NULL     |       |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM movie LIMIT 5;  
+-----+-----+-----+  
| id | name                | year |  
+-----+-----+-----+  
| 1  | Toy Story           | 1995 |  
| 2  | Jumanji             | 1995 |  
| 3  | Grumpier Old Men    | 1995 |  
| 4  | Waiting to Exhale   | 1995 |  
| 5  | Father of the Bride Part II | 1995 |  
+-----+-----+-----+  
5 rows in set (0.02 sec)  
  
mysql> █
```

Review the Database Tables

- Review the structure and contents of the *movierating* table:



A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows two MySQL queries and their results.

```
mysql> SELECT * FROM movie LIMIT 5;
```

id	name	year
1	Toy Story	1995
2	Jumanji	1995
3	Grumpier Old Men	1995
4	Waiting to Exhale	1995
5	Father of the Bride Part II	1995

```
5 rows in set (0.02 sec)
```



```
mysql> DESCRIBE movierating;
```

Field	Type	Null	Key	Default	Extra
userid	int(11)	NO	PRI	NULL	
movieid	int(11)	NO	PRI	NULL	
rating	tinyint(4)	NO		NULL	

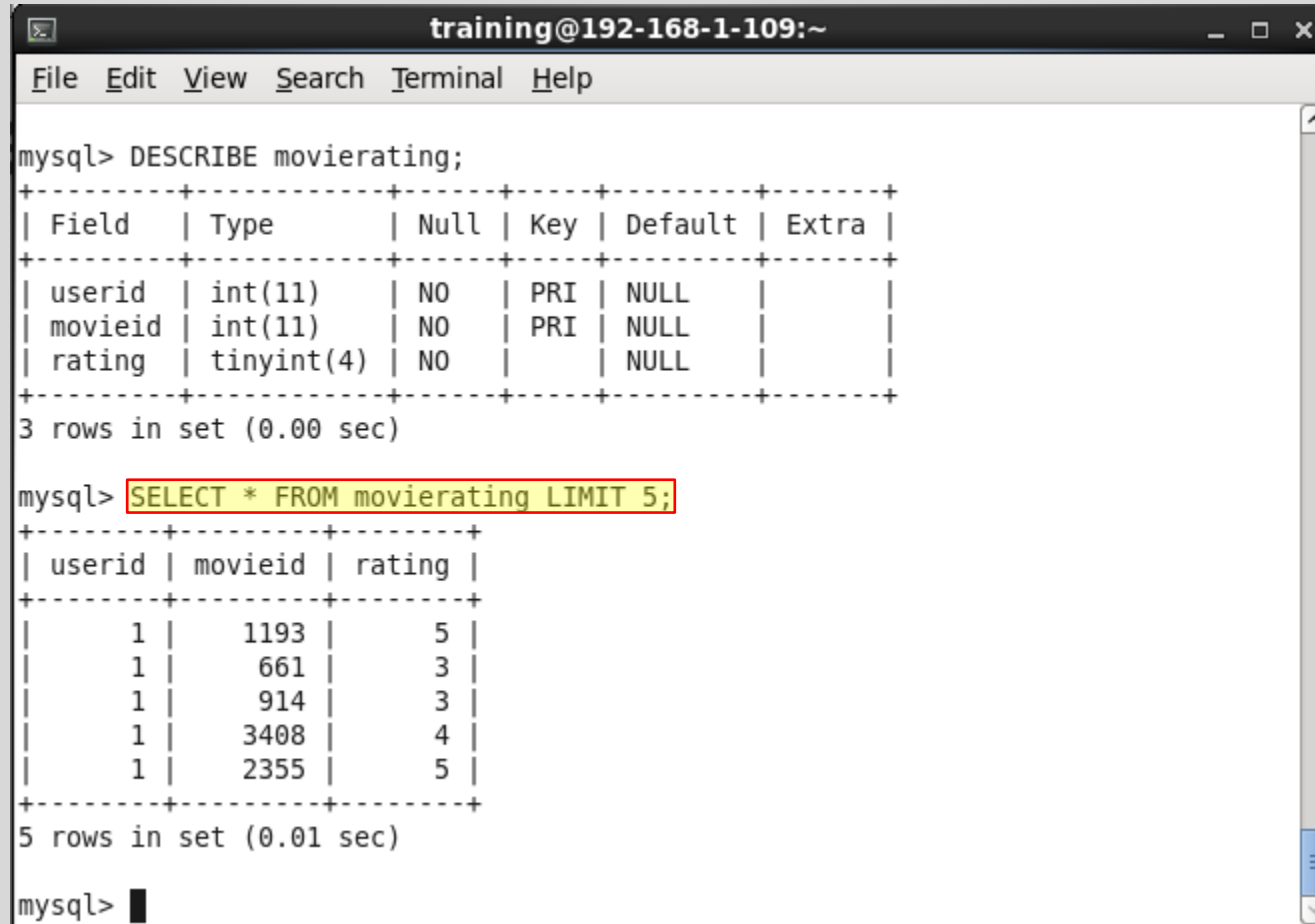
```
3 rows in set (0.00 sec)
```



```
mysql> █
```

Review the Database Tables

- Review the structure and contents of the *movierating* table:



```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
mysql> DESCRIBE movierating;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| userid | int(11)   | NO   | PRI | NULL    |      |  
| movieid | int(11)   | NO   | PRI | NULL    |      |  
| rating | tinyint(4) | NO   |     | NULL    |      |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM movierating LIMIT 5;  
+-----+-----+-----+  
| userid | movieid | rating |  
+-----+-----+-----+  
| 1      | 1193    | 5      |  
| 1      | 661     | 3      |  
| 1      | 914     | 3      |  
| 1      | 3408    | 4      |  
| 1      | 2355    | 5      |  
+-----+-----+-----+  
5 rows in set (0.01 sec)  
  
mysql> █
```

Review the Database Tables

- Exit mysql:

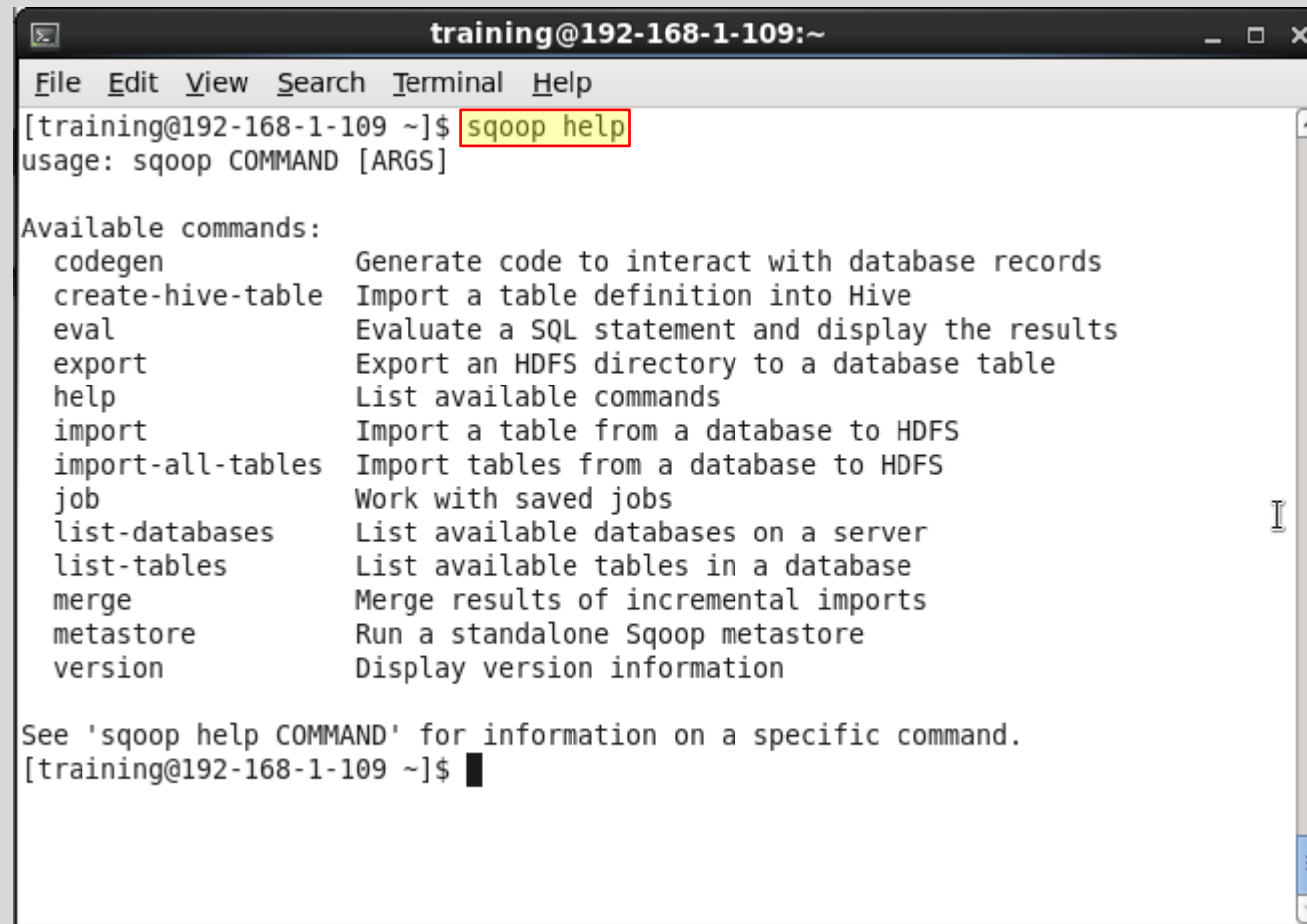
```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
+-----+-----+-----+-----+-----+  
| Field | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+  
| userid | int(11)    | NO   | PRI | NULL     |       |  
| movieid | int(11)    | NO   | PRI | NULL     |       |  
| rating  | tinyint(4) | NO   |     | NULL     |       |  
+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM movierating LIMIT 5;  
+-----+-----+-----+  
| userid | movieid | rating |  
+-----+-----+-----+  
| 1      | 1193    | 5      |  
| 1      | 661     | 3      |  
| 1      | 914     | 3      |  
| 1      | 3408    | 4      |  
| 1      | 2355    | 5      |  
+-----+-----+-----+  
5 rows in set (0.01 sec)  
  
mysql> quit  
Bye  
[training@192-168-1-109 ~]$
```

Import with Sqoop

- You invoke Sqoop on the command line to perform several commands.
- With it you can connect to your database server to list the databases (schemas) to which you have access, and list the tables available for loading.
- For database access, you provide a connect string to identify the server, and –if required – your username and password.

Import with Sqoop

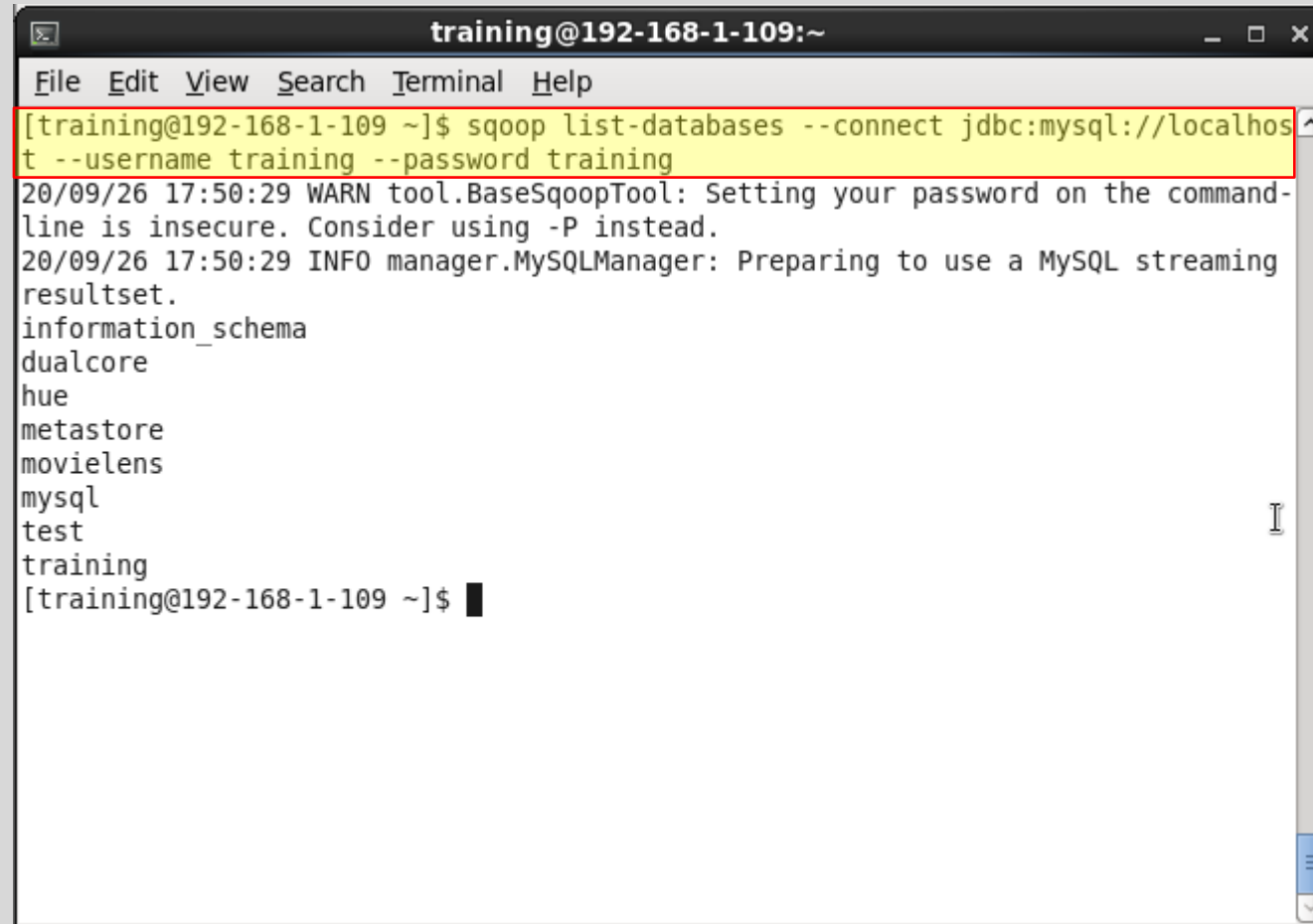
- Show the commands available in Sqoop:



```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop help  
usage: sqoop COMMAND [ARGS]  
  
Available commands:  
codegen          Generate code to interact with database records  
create-hive-table Import a table definition into Hive  
eval             Evaluate a SQL statement and display the results  
export           Export an HDFS directory to a database table  
help             List available commands  
import           Import a table from a database to HDFS  
import-all-tables Import tables from a database to HDFS  
job              Work with saved jobs  
list-databases   List available databases on a server  
list-tables      List available tables in a database  
merge            Merge results of incremental imports  
metastore        Run a standalone Sqoop metastore  
version          Display version information  
  
See 'sqoop help COMMAND' for information on a specific command.  
[training@192-168-1-109 ~]$
```

Import with Sqoop

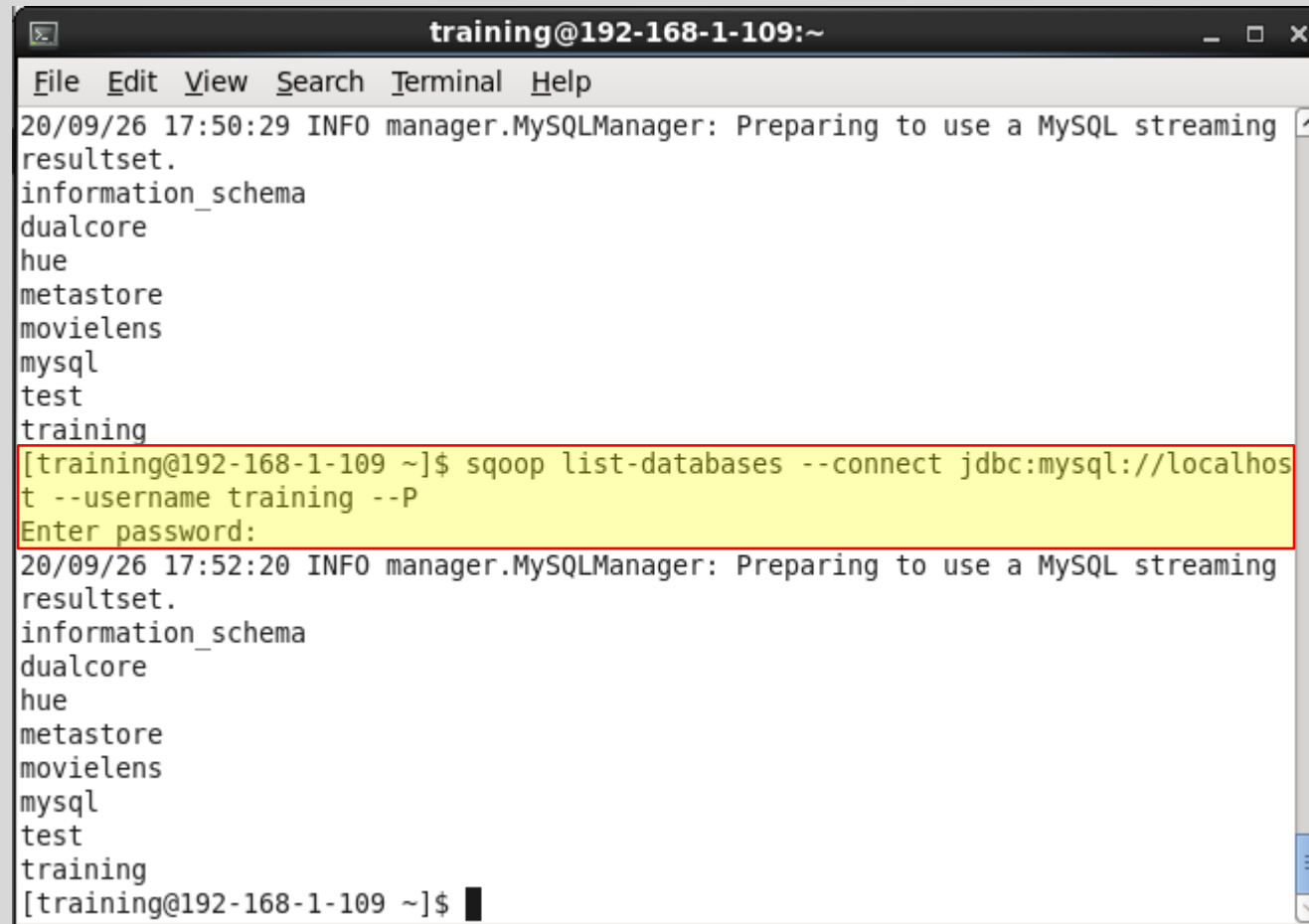
- List the database (schemas) in your database server:

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[training@192-168-1-109 ~]\$ sqoop list-databases --connect jdbc:mysql://localhost --username training --password training' is highlighted in yellow. The output shows a list of databases: information_schema, dualcore, hue, metastore, movielens, mysql, test, training. The prompt '[training@192-168-1-109 ~]\$' is followed by a cursor.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop list-databases --connect jdbc:mysql://localhost  
t --username training --password training  
20/09/26 17:50:29 WARN tool.BaseSqoopTool: Setting your password on the command-  
line is insecure. Consider using -P instead.  
20/09/26 17:50:29 INFO manager.MySQLManager: Preparing to use a MySQL streaming  
resultset.  
information_schema  
dualcore  
hue  
metastore  
movielens  
mysql  
test  
training  
[training@192-168-1-109 ~]$
```

Import with Sqoop

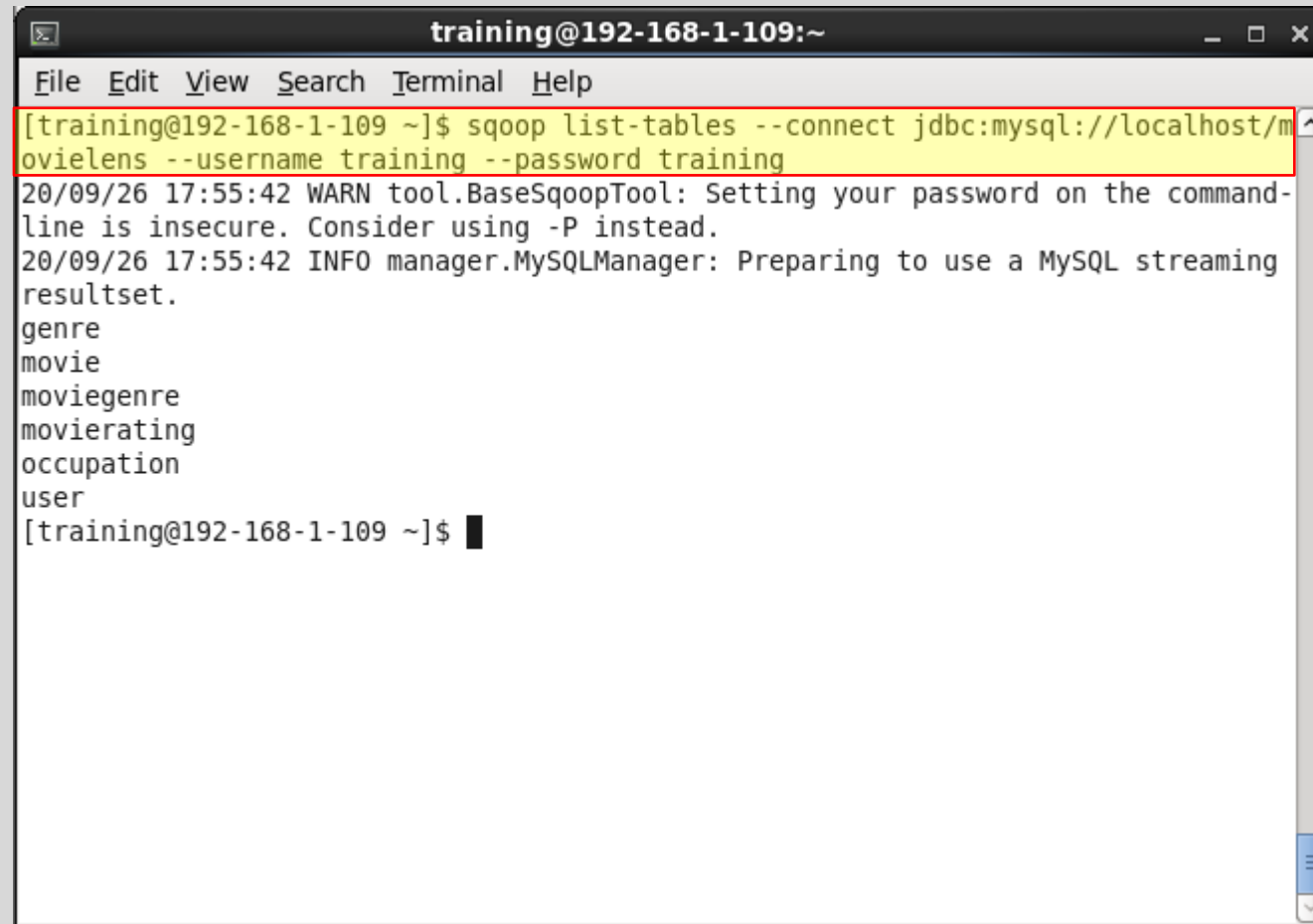
- You can also use `-P` to replace `-password`. This will hide your password when entering.

A terminal window titled 'training@192-168-1-109:~' showing the execution of the 'sqoop list-databases' command. The command is highlighted with a red box. The output lists several databases: information_schema, dualcore, hue, metastore, movielens, mysql, test, and training. The terminal also shows status messages from the MySQL manager.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
20/09/26 17:50:29 INFO manager.MySQLManager: Preparing to use a MySQL streaming  
resultset.  
information_schema  
dualcore  
hue  
metastore  
movielens  
mysql  
test  
training  
[training@192-168-1-109 ~]$ sqoop list-databases --connect jdbc:mysql://localhos  
t --username training --P  
Enter password:  
20/09/26 17:52:20 INFO manager.MySQLManager: Preparing to use a MySQL streaming  
resultset.  
information_schema  
dualcore  
hue  
metastore  
movielens  
mysql  
test  
training  
[training@192-168-1-109 ~]$
```

Import with Sqoop

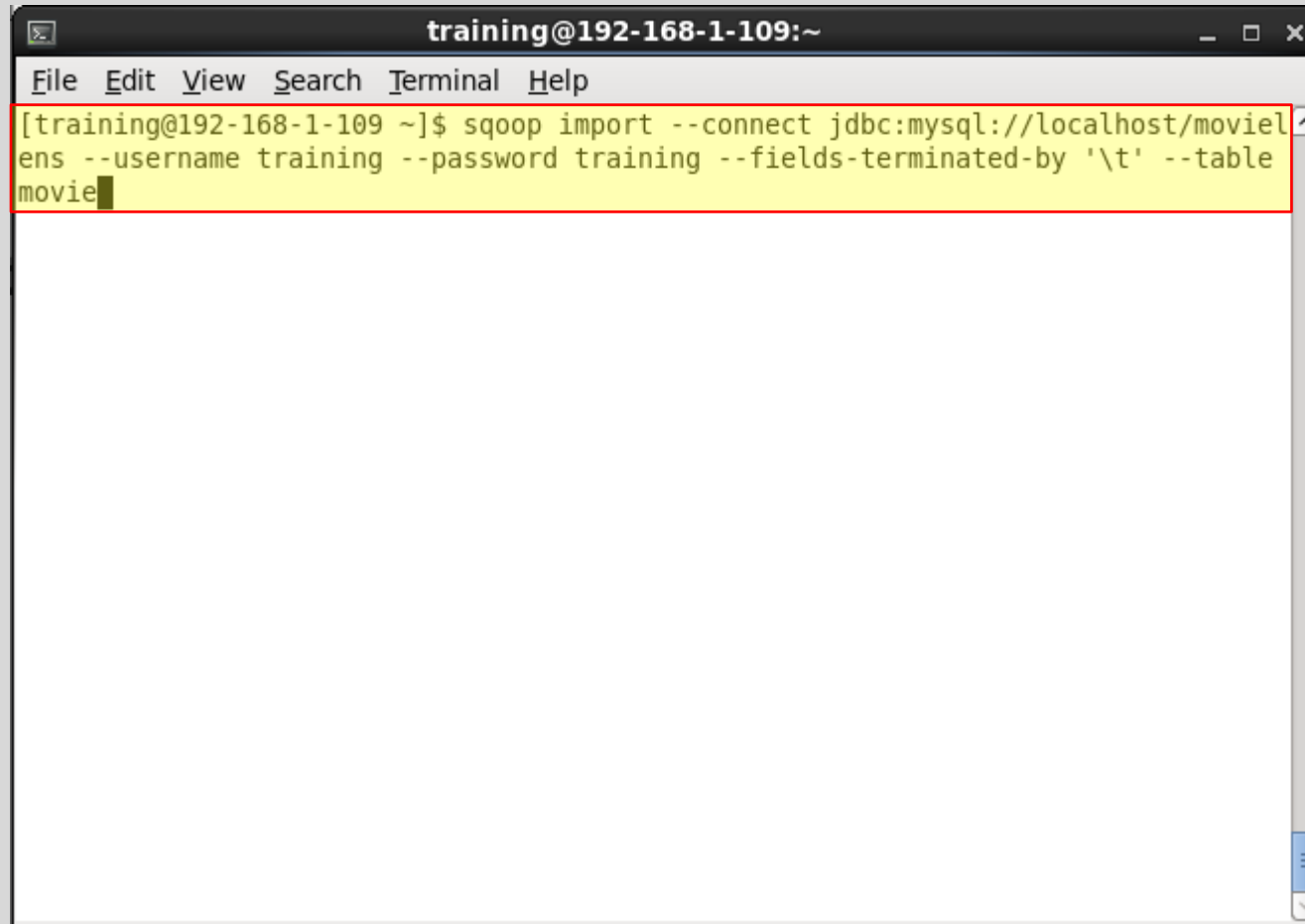
- List the tables in the *movielens* database:

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[training@192-168-1-109 ~]\$ sqoop list-tables --connect jdbc:mysql://localhost/movielens --username training --password training' is highlighted in yellow. The output shows a warning about password security, followed by a list of tables: genre, movie, moviegenre, movierating, occupation, and user. The prompt returns to '[training@192-168-1-109 ~]\$' with a cursor.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop list-tables --connect jdbc:mysql://localhost/movielens --username training --password training  
20/09/26 17:55:42 WARN tool.BaseSqoopTool: Setting your password on the command-line is insecure. Consider using -P instead.  
20/09/26 17:55:42 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.  
genre  
movie  
moviegenre  
movierating  
occupation  
user  
[training@192-168-1-109 ~]$
```

Import with Sqoop

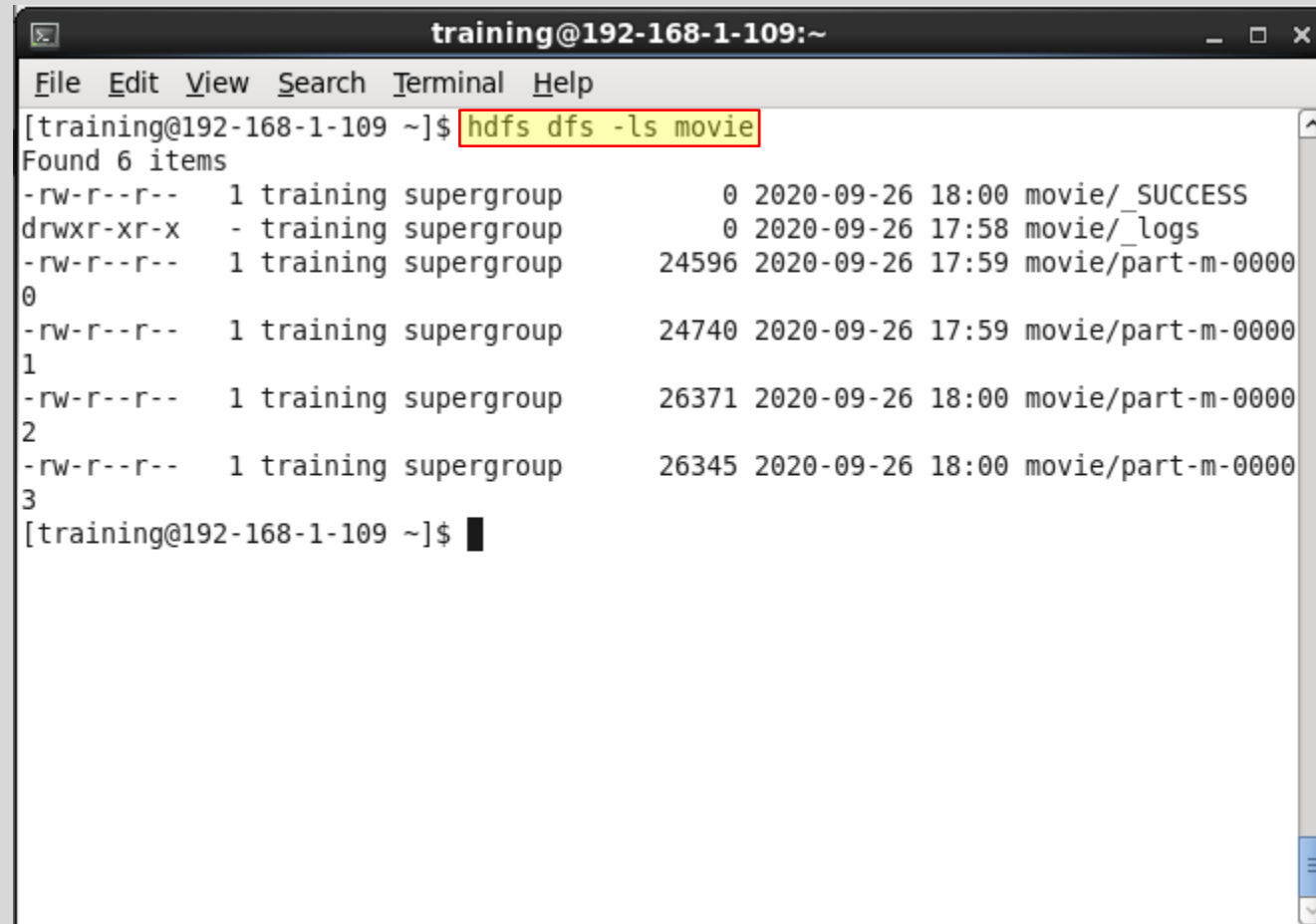
- Import the *movie* table into Hadoop:

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[training@192-168-1-109 ~]\$ sqoop import --connect jdbc:mysql://localhost/movies --username training --password training --fields-terminated-by '\\t' --table movie' is entered and highlighted with a red box. The cursor is at the end of the command.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop import --connect jdbc:mysql://localhost/movies --username training --password training --fields-terminated-by '\\t' --table movie
```

Import with Sqoop

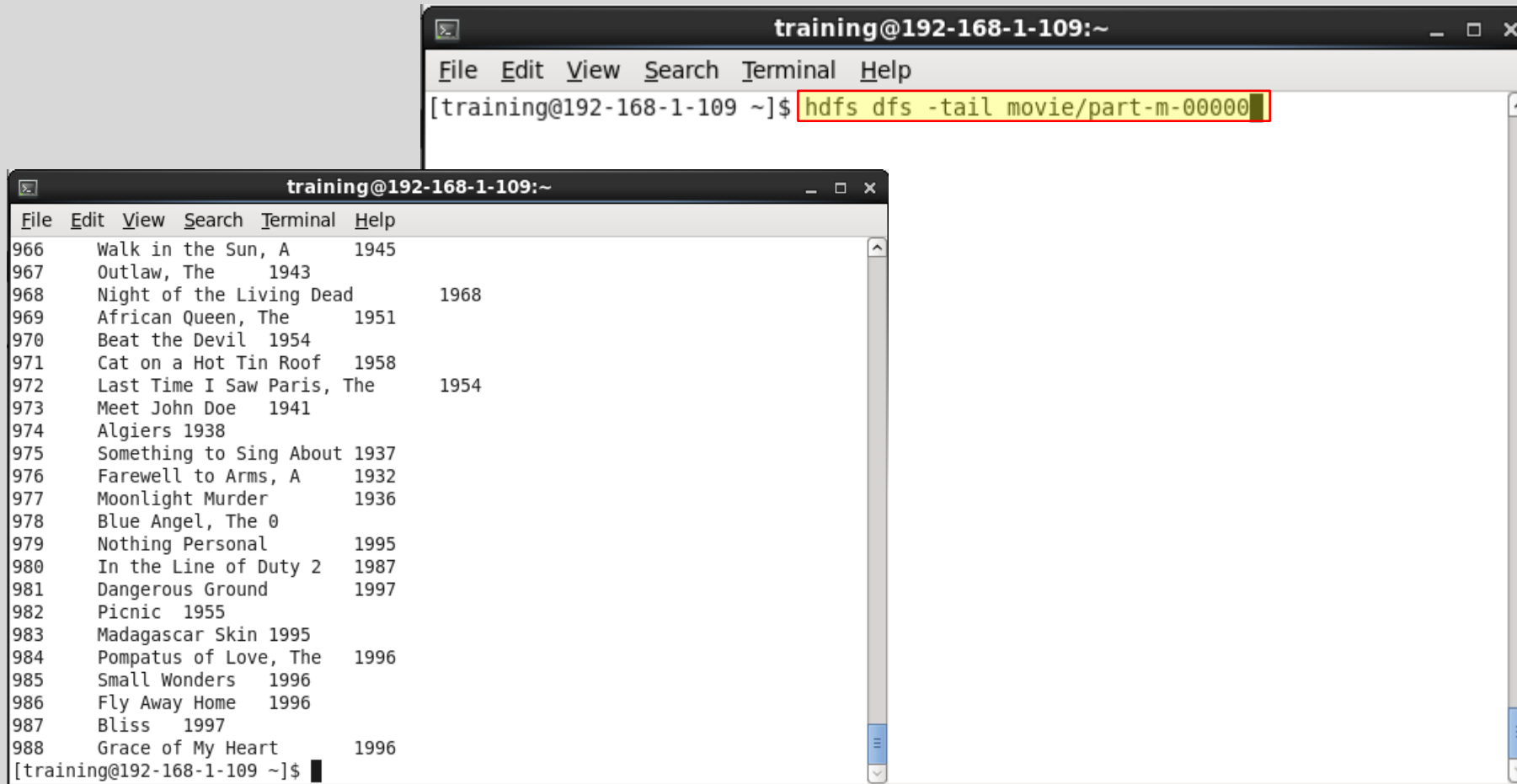
- Verify that the command has worked.



```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ hdfs dfs -ls movie  
Found 6 items  
-rw-r--r-- 1 training supergroup 0 2020-09-26 18:00 movie/_SUCCESS  
drwxr-xr-x - training supergroup 0 2020-09-26 17:58 movie/_logs  
-rw-r--r-- 1 training supergroup 24596 2020-09-26 17:59 movie/part-m-0000  
0  
-rw-r--r-- 1 training supergroup 24740 2020-09-26 17:59 movie/part-m-0000  
1  
-rw-r--r-- 1 training supergroup 26371 2020-09-26 18:00 movie/part-m-0000  
2  
-rw-r--r-- 1 training supergroup 26345 2020-09-26 18:00 movie/part-m-0000  
3  
[training@192-168-1-109 ~]$
```

Import with Sqoop

- Verify that the command has worked.



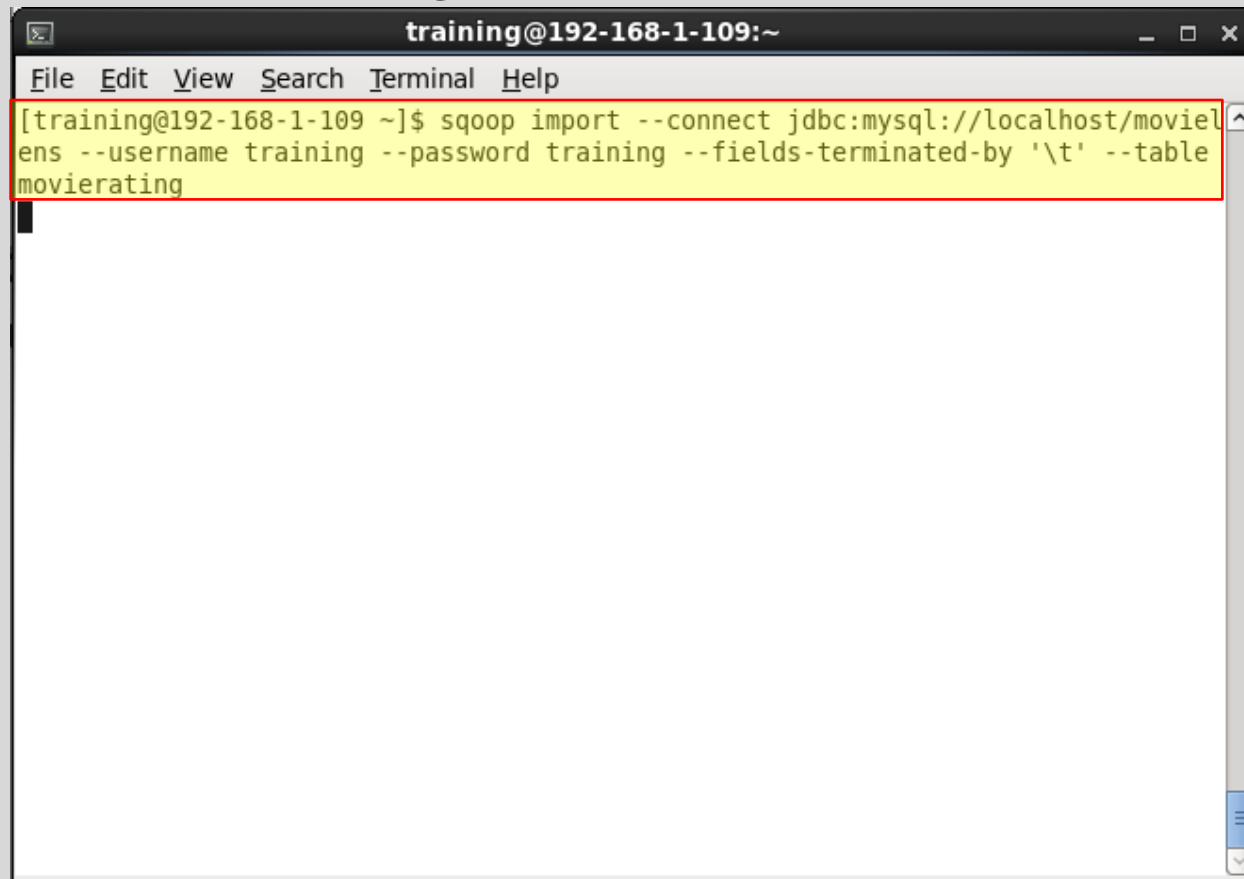
The image displays two terminal windows from a user named 'training' on a machine with IP '192-168-1-109'. The top window shows the command `hdfs dfs -tail movie/part-m-00000` being entered at the prompt. The bottom window shows the output of this command, which is a list of movie titles and their release years, each preceded by a line number (966 to 988).

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ hdfs dfs -tail movie/part-m-00000
```

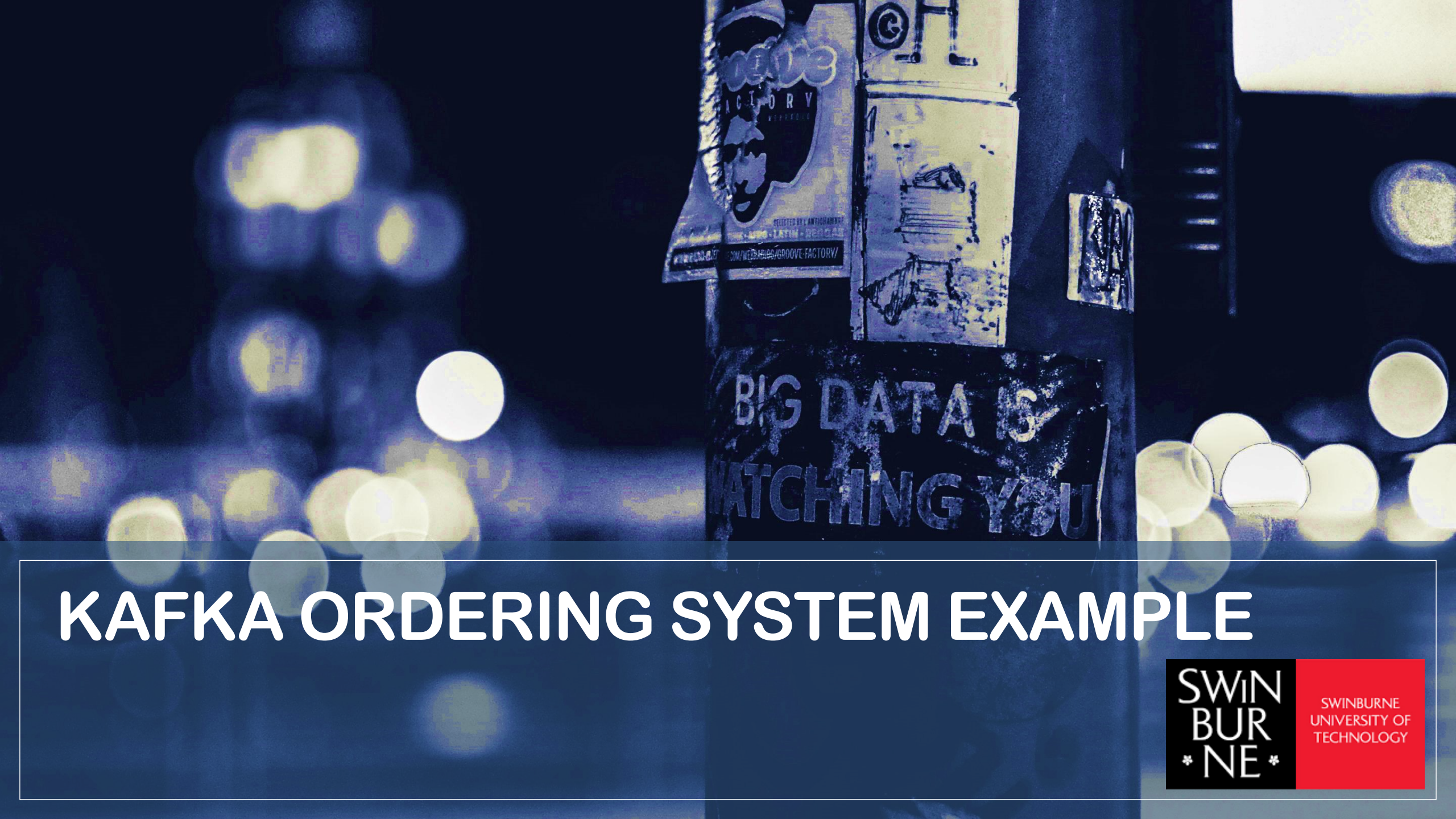
```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
966 Walk in the Sun, A 1945  
967 Outlaw, The 1943  
968 Night of the Living Dead 1968  
969 African Queen, The 1951  
970 Beat the Devil 1954  
971 Cat on a Hot Tin Roof 1958  
972 Last Time I Saw Paris, The 1954  
973 Meet John Doe 1941  
974 Algiers 1938  
975 Something to Sing About 1937  
976 Farewell to Arms, A 1932  
977 Moonlight Murder 1936  
978 Blue Angel, The 0  
979 Nothing Personal 1995  
980 In the Line of Duty 2 1987  
981 Dangerous Ground 1997  
982 Picnic 1955  
983 Madagascar Skin 1995  
984 Pompatus of Love, The 1996  
985 Small Wonders 1996  
986 Fly Away Home 1996  
987 Bliss 1997  
988 Grace of My Heart 1996  
[training@192-168-1-109 ~]$
```

Import with Sqoop

- Import the *movierating* table into Hadoop by repeating the last two steps, but for the *movierating* table.

A terminal window titled 'training@192-168-1-109:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[training@192-168-1-109 ~]\$ sqoop import --connect jdbc:mysql://localhost/movielens --username training --password training --fields-terminated-by '\\t' --table movierating' is entered and highlighted with a red box. A cursor is visible at the end of the command line.

```
training@192-168-1-109:~  
File Edit View Search Terminal Help  
[training@192-168-1-109 ~]$ sqoop import --connect jdbc:mysql://localhost/movielens --username training --password training --fields-terminated-by '\\t' --table movierating
```



KAFKA ORDERING SYSTEM EXAMPLE



USING KAFKA TO SIMULATE AN ORDERING SYSTEM

Files and Directories Used in this Exercise

Eclipse project: kafka_ordering_system

Java files:

- CafeMain.java
- CashierMain.java
- DriveThruMain.java
- KitchenMain.java

Exercise directory: ~/workspace/kafka_ordering_system

Description

- In this part of week 08 lab, you'll implement an ordering system within a restaurant using Apache Kafka
- This Kafka architecture consists of two publishers, two subscribers, one broker and two topics
- One publisher, "drive_thru" and one subscriber, "kitchen", have already been provided, and you'll implement the remaining two, "café" and "cashier"

Note: An automatic order generation feature has already been provided to you. When there is a need to publish order-related messages, please refer to the method used within the "drive_thru" component.

Review the provided publisher and subscriber

- Before proceeding with the implementation of the remaining components, let's take a closer look at the provided publisher, “drive_thru”, and subscriber, “kitchen”
- Drive_thru, publisher:
 - Publishes messages to both the “food” and “drink” topics
- Kitchen, subscriber:
 - Subscribes to the “food” topic

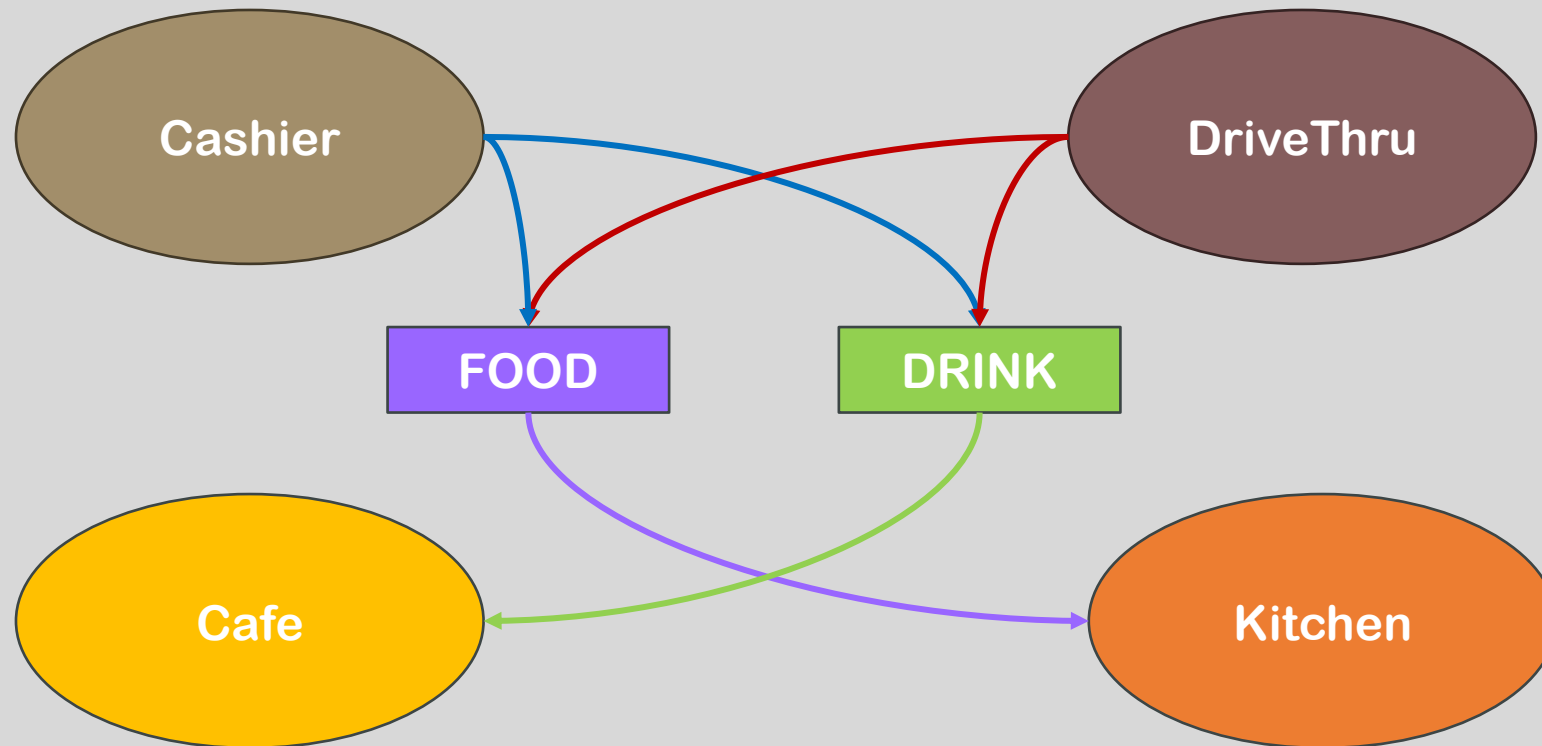
Implementing Cashier and Cafe

- The next task is to implement the “cashier” and the “Cafe”.
- **Cafe Component:**

The “café” receives messages from both “cashier” and “drive_thru”.
Its processes these messages and prepare beverages accordingly.
- **Cashier Component:**

The "cashier" transmits order messages onto the designated topic.
- You need to determine which one serves as the publisher and which one as the subscriber

System Diagram

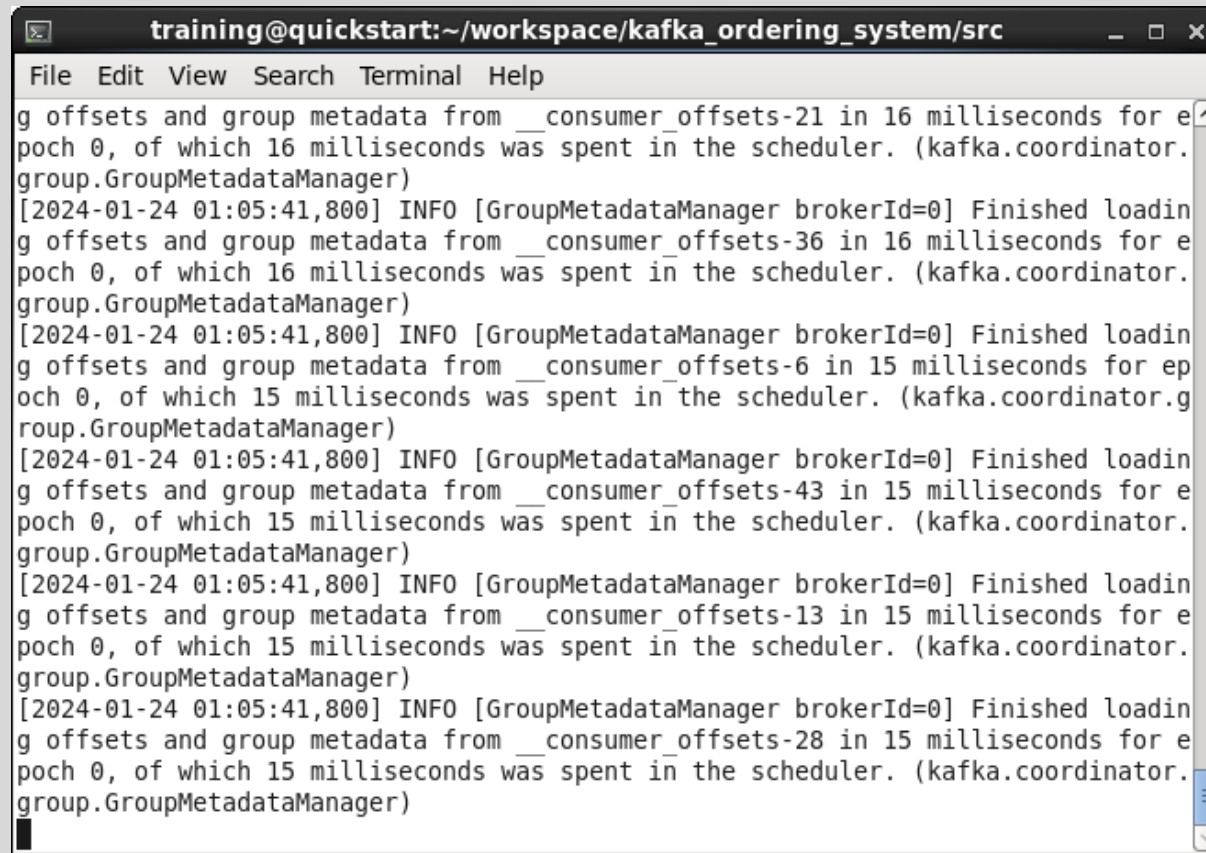


Run the broker

- Open a terminal
- Navigate to the project directory with
`'cd ~/workspace/kafka_ordering_system/src/'`
- Run the broker with `'sh run_broker.sh'`

Run the broker

After a successful execution, you will see the following content

A terminal window titled 'training@quickstart:~/workspace/kafka_ordering_system/src' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal displays a series of log messages from the 'kafka.coordinator.group.GroupMetadataManager' indicating that it has finished loading offsets and group metadata for various consumer groups (e.g., __consumer_offsets-21, __consumer_offsets-36, __consumer_offsets-6, __consumer_offsets-43, __consumer_offsets-13, __consumer_offsets-28) for epoch 0. Each message includes a timestamp of [2024-01-24 01:05:41,800] and the broker ID 0. The messages specify the time spent in the scheduler (16 or 15 milliseconds).

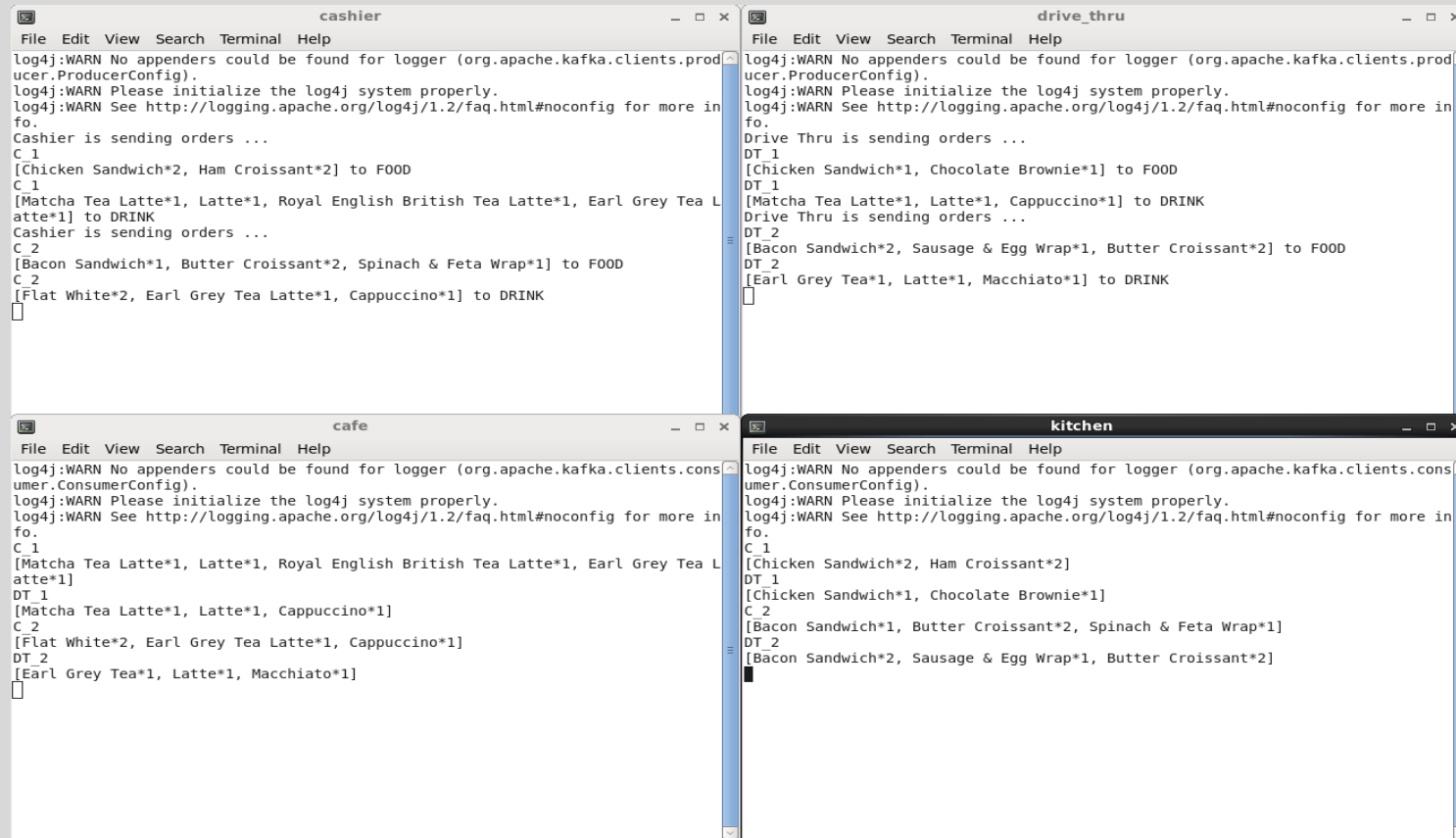
```
training@quickstart:~/workspace/kafka_ordering_system/src
File Edit View Search Terminal Help
g offsets and group metadata from __consumer_offsets-21 in 16 milliseconds for e
poch 0, of which 16 milliseconds was spent in the scheduler. (kafka.coordinator.
group.GroupMetadataManager)
[2024-01-24 01:05:41,800] INFO [GroupMetadataManager brokerId=0] Finished loadin
g offsets and group metadata from __consumer_offsets-36 in 16 milliseconds for e
poch 0, of which 16 milliseconds was spent in the scheduler. (kafka.coordinator.
group.GroupMetadataManager)
[2024-01-24 01:05:41,800] INFO [GroupMetadataManager brokerId=0] Finished loadin
g offsets and group metadata from __consumer_offsets-6 in 15 milliseconds for ep
och 0, of which 15 milliseconds was spent in the scheduler. (kafka.coordinator.g
roup.GroupMetadataManager)
[2024-01-24 01:05:41,800] INFO [GroupMetadataManager brokerId=0] Finished loadin
g offsets and group metadata from __consumer_offsets-43 in 15 milliseconds for e
poch 0, of which 15 milliseconds was spent in the scheduler. (kafka.coordinator.
group.GroupMetadataManager)
[2024-01-24 01:05:41,800] INFO [GroupMetadataManager brokerId=0] Finished loadin
g offsets and group metadata from __consumer_offsets-13 in 15 milliseconds for e
poch 0, of which 15 milliseconds was spent in the scheduler. (kafka.coordinator.
group.GroupMetadataManager)
[2024-01-24 01:05:41,800] INFO [GroupMetadataManager brokerId=0] Finished loadin
g offsets and group metadata from __consumer_offsets-28 in 15 milliseconds for e
poch 0, of which 15 milliseconds was spent in the scheduler. (kafka.coordinator.
group.GroupMetadataManager)
```

Compile and execute the code of ordering system

- Open a terminal
- Navigate to the project directory with
`'cd ~/workspace/kafka_ordering_system/src/'`
- Compile the implemented components with
`'sh compile_all_apps.sh'`
- Run the implemented components with
`'sh run_all_apps.sh'`

Run the ordering system

- After a successful execution, you will see something similar to the following content



The image displays four terminal windows arranged in a 2x2 grid, each showing the output of a different component of the ordering system. Each window has a title bar and a menu bar (File, Edit, View, Search, Terminal, Help). The output in each window includes log4j warnings about missing appenders and logger configurations, followed by a series of messages indicating the flow of orders between the cashier, drive-thru, cafe, and kitchen.

```
cashier
log4j:WARN No appenders could be found for logger (org.apache.kafka.clients.producer.ProducerConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Cashier is sending orders ...
C 1
[Chicken Sandwich*2, Ham Croissant*2] to FOOD
C 1
[Matcha Tea Latte*1, Latte*1, Royal English British Tea Latte*1, Earl Grey Tea Latte*1] to DRINK
Cashier is sending orders ...
C 2
[Bacon Sandwich*1, Butter Croissant*2, Spinach & Feta Wrap*1] to FOOD
C 2
[Flat White*2, Earl Grey Tea Latte*1, Cappuccino*1] to DRINK

drive_thru
log4j:WARN No appenders could be found for logger (org.apache.kafka.clients.producer.ProducerConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Drive Thru is sending orders ...
DT 1
[Chicken Sandwich*1, Chocolate Brownie*1] to FOOD
DT 1
[Matcha Tea Latte*1, Latte*1, Cappuccino*1] to DRINK
Drive Thru is sending orders ...
DT 2
[Bacon Sandwich*2, Sausage & Egg Wrap*1, Butter Croissant*2] to FOOD
DT 2
[Earl Grey Tea*1, Latte*1, Macchiato*1] to DRINK

cafe
log4j:WARN No appenders could be found for logger (org.apache.kafka.clients.consumer.ConsumerConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
C 1
[Matcha Tea Latte*1, Latte*1, Royal English British Tea Latte*1, Earl Grey Tea Latte*1]
DT 1
[Matcha Tea Latte*1, Latte*1, Cappuccino*1]
C 2
[Flat White*2, Earl Grey Tea Latte*1, Cappuccino*1]
DT 2
[Earl Grey Tea*1, Latte*1, Macchiato*1]

kitchen
log4j:WARN No appenders could be found for logger (org.apache.kafka.clients.consumer.ConsumerConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
C 1
[Chicken Sandwich*2, Ham Croissant*2]
DT 1
[Chicken Sandwich*1, Chocolate Brownie*1]
C 2
[Bacon Sandwich*1, Butter Croissant*2, Spinach & Feta Wrap*1]
DT 2
[Bacon Sandwich*2, Sausage & Egg Wrap*1, Butter Croissant*2]
```

Clean-up

- Now we reached the end of the ordering system
- Stop the producers and the consumers with Ctrl-C
- Stop the broker with Ctrl-C
- Clean up the kafka logs with `'rm -rf /tmp/kafka-logs'`